

Lecture 3

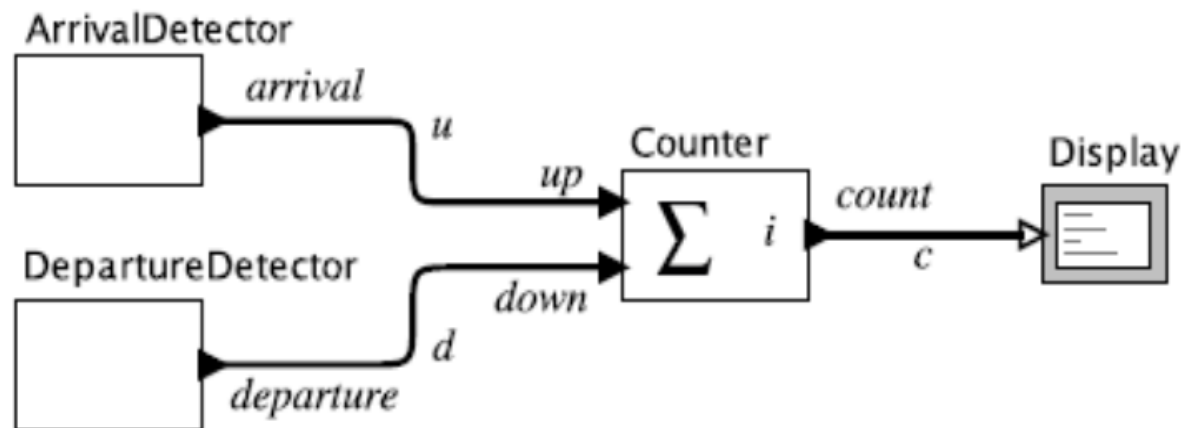
Discrete Dynamics

Discrete Systems

- Models of embedded systems include both discrete and continuous components.
- In this chapter, we study how state machines can be used to model discrete dynamics.
- A discrete system operates in a sequence of discrete steps and is said to have discrete dynamics.
- Some systems are inherently discrete.

Example 3.1:

- Consider a system that counts the number of cars that enter and leave a parking garage in order to keep track of how many cars are in the garage at any time.
- It could be modeled using Actor model as shown in Figure.



Cont.,

- Where: The inputs to the counter are discrete (Pure) signals.
- The input to the Counter is a pair of discrete signals that at certain times have an event (are present), and at other times have no event (are absent).
- The output also is a discrete signal that, when an input is present, has a value that is a natural number, and at other times is absent

$$u: \mathbb{R} \rightarrow \{absent, present\}.$$

$$c: \mathbb{R} \rightarrow \{absent\} \cup \mathbb{Z}.$$

Dynamics of a discrete system

- The dynamics of a discrete system can be described as a sequence of steps that we call reactions, each of which we assume to be instantaneous.
- Reactions of a discrete system are triggered by the environment in which the discrete system operates.
- In the case of the example of garage, reactions of the Counter actor are triggered when one or more input events are present (Event triggered).

The Notion of State

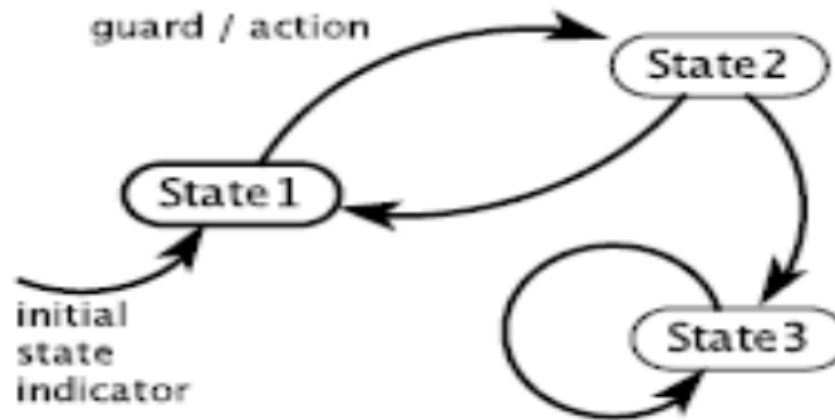
- The state of a system is its condition at a particular point in time.
- In general, the state affects how the system reacts to inputs.
- A practical parking garage has a finite and non-negative number M of spaces, so the state space for the Counter actor used in this way will be:
$$\text{States} = \{0; 1; 2; \dots; M\}$$
- The state space for the garage counter is finite.
- Discrete models with finite state spaces are called finite-state machines (FSMs).

Finite-State Machines

- A state machine is a model of a system with discrete dynamics that at each reaction maps valuations of the inputs to valuations of the outputs, where the map may depend on its current state.
- A finite-state machine (FSM) is a state machine where the set of possible states is finite.
- At the beginning of each sequence of reactions, there is an initial state, State1, indicated in the diagram by a dangling arrow into it.

Transitions

- **Transitions** between states govern the discrete dynamics of the state machine and the mapping of input valuations to output valuations.



Cont.,

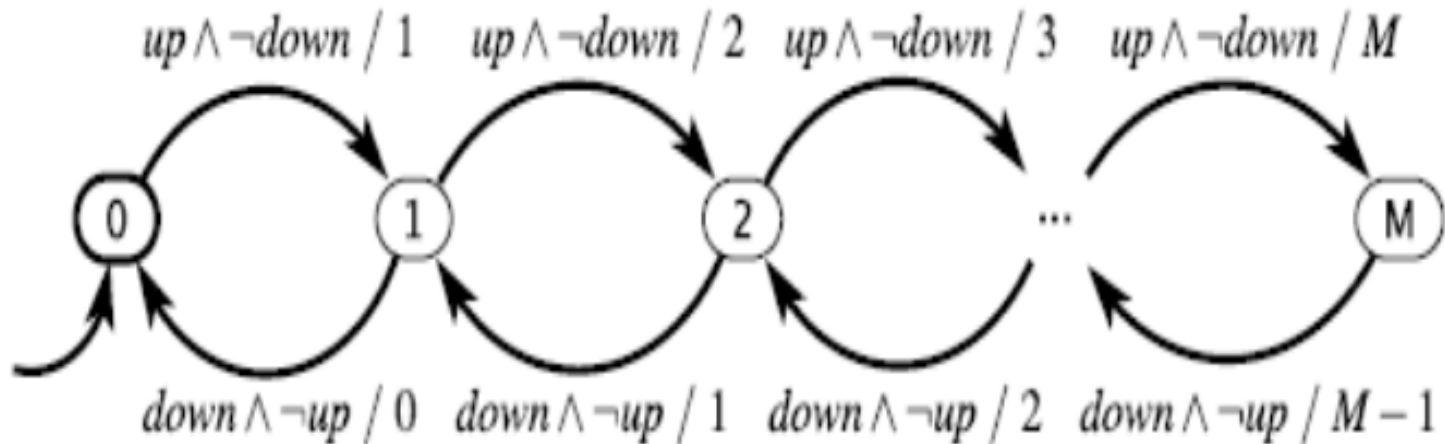
- In Figure 3.3, the transition from State1 to State2 is labeled with “guard / action.”
- The guard (input) determines whether the transition may be taken on a reaction.
- The action specifies what outputs are produced on each reaction.
- A guard is a predicate (a Boolean-valued expression) that evaluates to true when the transition should be taken.

Example 1

- FSM model for the garage counter (Mealy Machine)

inputs: $up, down$: pure

output: $count : \{0, \dots, M\}$



Example 2

- A major use of energy worldwide is in heating, ventilation, and air conditioning (HVAC) systems.
- Consider a thermostat modeled by an FSM with States = {heating, cooling}.

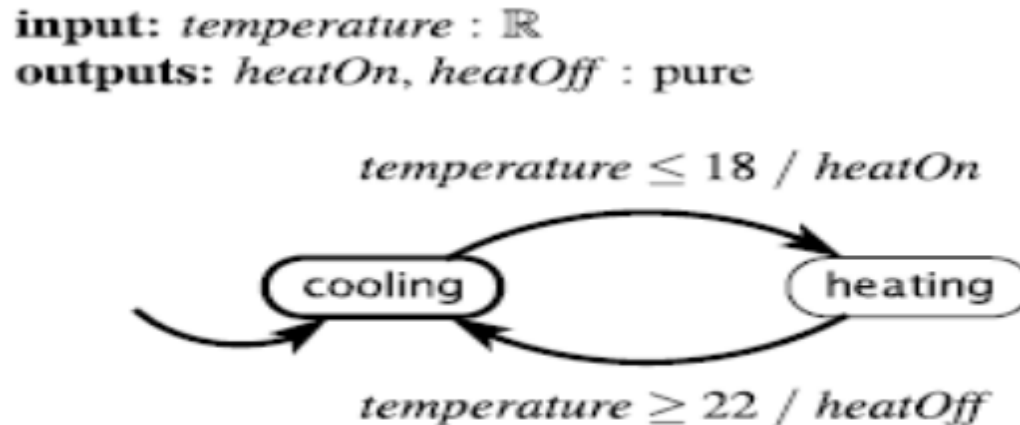


Figure 3.5: A model of a thermostat with hysteresis.

Update Functions

- The graphical notation for FSMs defines a specific mathematical model of the dynamics of a state machine.
- In such a mathematical notation, a finite-state machine is a five-tuple
(States; Inputs; Outputs; update; initial State)
- Where, $\text{update} : \text{States} \times \text{Inputs} \rightarrow \text{States} \times \text{Outputs}$ is an update function, mapping a state and an input valuation to a next state and an output valuation;

Cont.,

- The dynamics of the state machine are given by the following equation:

$$(s(n + 1); y(n)) = \text{update}(s(n); x(n))$$

Where:

$S(n+1)$ is the next state

$S(n)$ is the current state

$Y(n)$ is the output from state n

$X(n)$ is the input to state n

Example

- The FSM for the garage counter can be mathematically represented as follows:

$$\begin{aligned} \text{States} &= \{0, 1, \dots, M\} \\ \text{Inputs} &= (\{up, down\} \rightarrow \{present, absent\}) \\ \text{Outputs} &= (\{count\} \rightarrow \{0, 1, \dots, M, absent\}) \\ \text{initialState} &= 0 \end{aligned}$$

The update function is given by

$$\text{update}(s, i) = \begin{cases} (s + 1, s + 1) & \text{if } s < M \\ & \wedge i(up) = present \\ & \wedge i(down) = absent \\ (s - 1, s - 1) & \text{if } s > 0 \\ & \wedge i(up) = absent \\ & \wedge i(down) = present \\ (s, absent) & \text{otherwise} \end{cases}$$

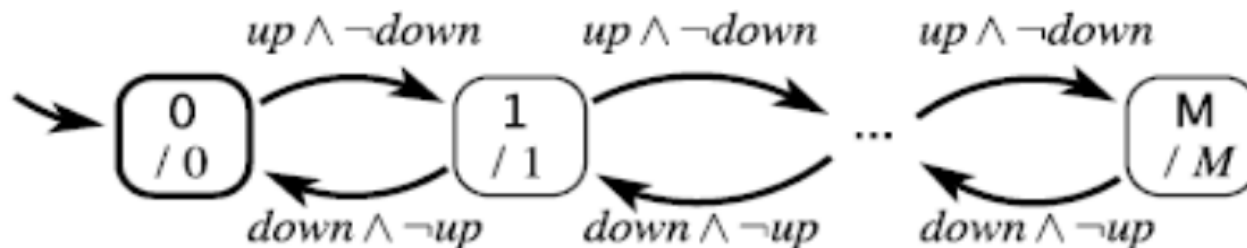
Determinacy

- Determinacy: A state machine is said to be deterministic if, for each state, there is at most one transition enabled by each input value.
- The formal definition of an FSM given above ensures that it is deterministic, since update is a one – to- one function, not a one-to many mapping.
- Note that a deterministic state machine is determinate, meaning that given the same inputs it will always produce the same outputs.

Moore Machines and Mealy Machines

- Mealy machines are characterized by producing outputs when a transition is taken.
- An alternative, known as a Moore machine, produces outputs when the machine is in a state, rather than when a transition is taken.
- That is, the output is defined by the current state rather than by the current transition.

inputs: *up, down*: pure
output: *count*: $\{0, \dots, M\}$



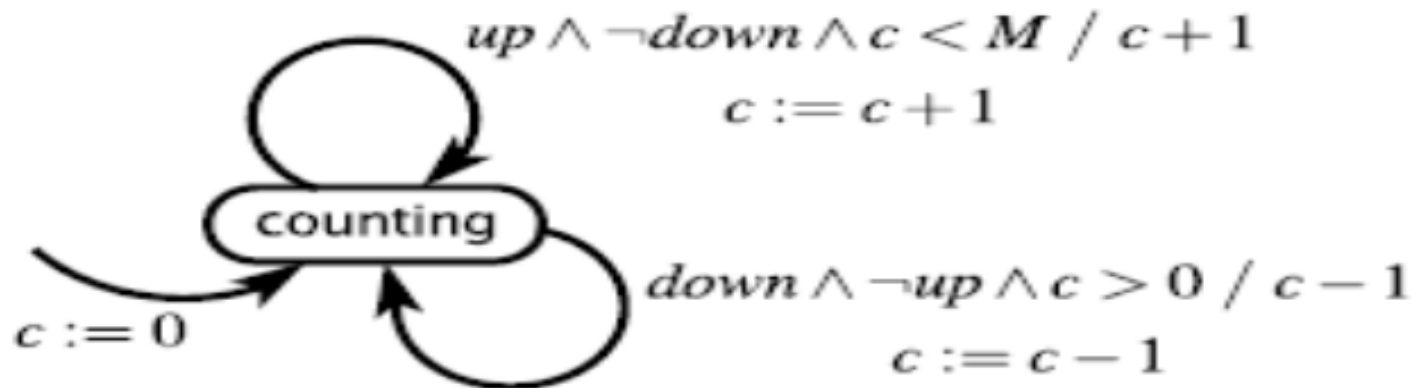
Extended State Machines

- When the number of states gets large, an extended state machine solves this problem by augmenting the FSM model with variables that may be read and written as part of taking a transition between states.

Example

- The garage counter can be represented more compactly by the extended state machine

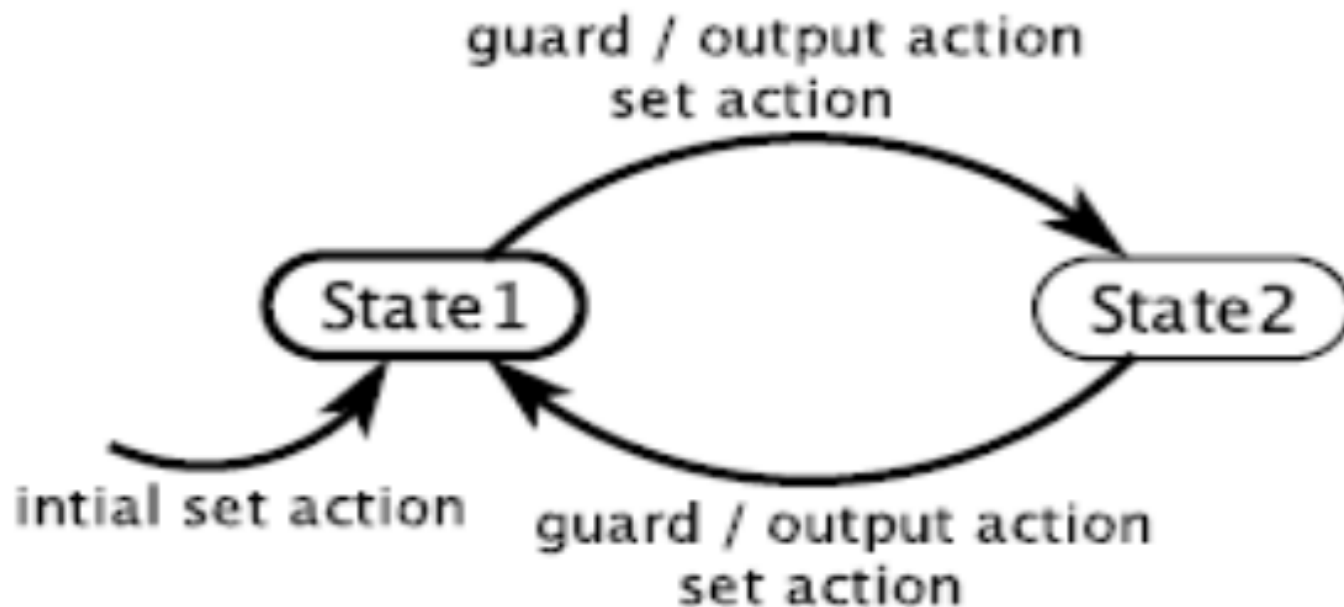
variable: $c: \{0, \dots, M\}$
inputs: $up, down$: pure
output: $count: \{0, \dots, M\}$



Extended state machine for the garage counter

General Notation for ESM

variable declaration(s)
input declaration(s)
output declaration(s)



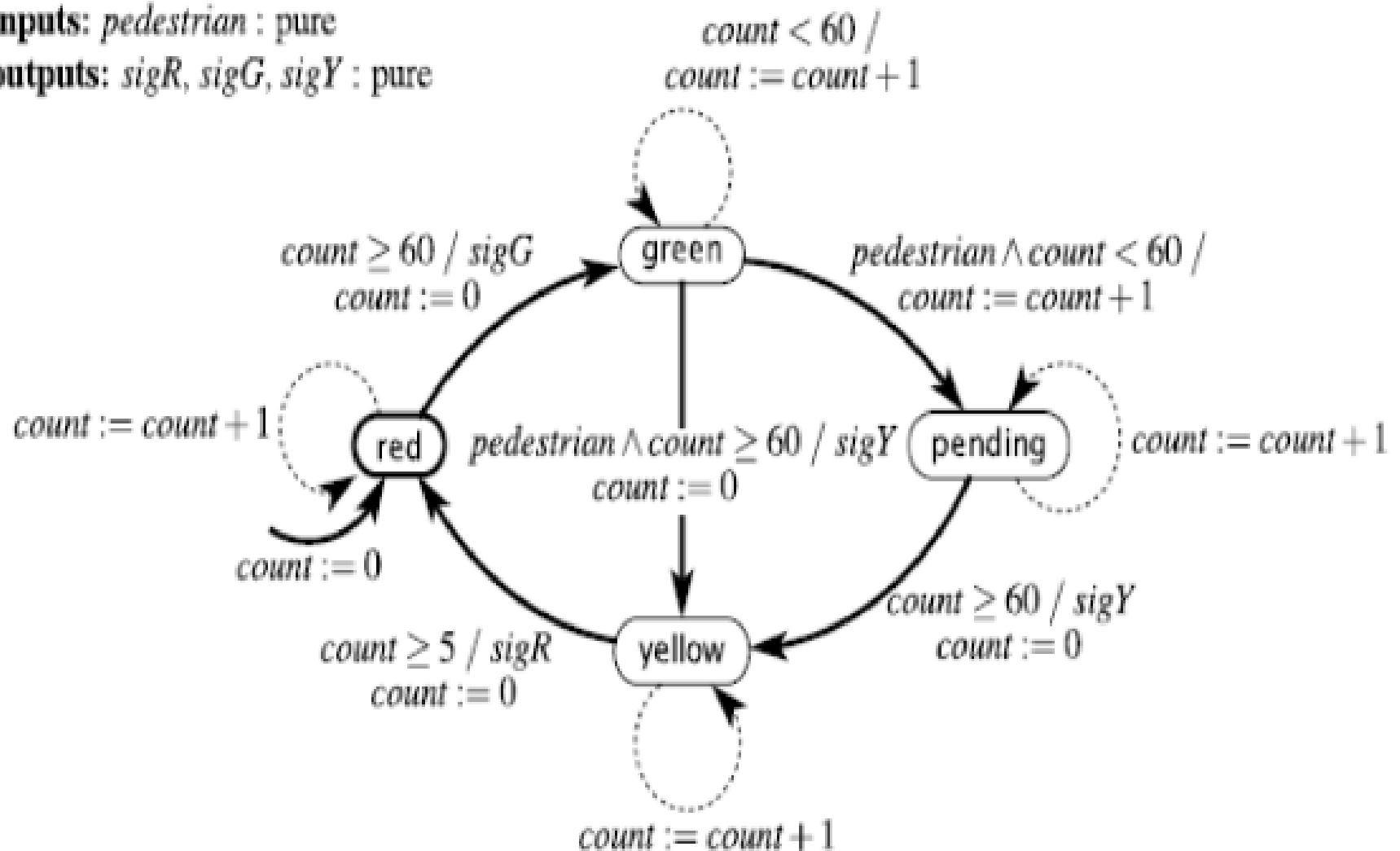
Example

- An extended state machine describing a traffic light at a pedestrian crosswalk.
- It starts in the red state and counts 60 seconds with the help of the variable count.
- It then transitions to green, where it will remain until the pure input pedestrian is present.
- That input could be generated, for example, by a pedestrian pushing a button to request a walk light.
- When pedestrian is present, the machine transitions to yellow if it has been in state green for at least 60 seconds.
- Otherwise, it transitions to pending, where it stays for the remainder of the 60 second interval.

variable: $count: \{0, \dots, 60\}$

inputs: $pedestrian : \text{pure}$

outputs: $sigR, sigG, sigY : \text{pure}$



Nondeterminism

- Most interesting state machines react to inputs and produce outputs. These inputs must come from somewhere, and the outputs must go somewhere.
- We refer to this “somewhere” as the environment of the state machine.
- If for any state of a state machine, there are two distinct transitions with guards that can evaluate to true in the same reaction, then the state machine is nondeterministic.
- A nondeterministic FSM specifies a family of possible reactions rather than a single reaction. Operationally, all reactions in the family are possible.

Formal Model

- Formally, a nondeterministic FSM is represented as a five-tuple, similar to a deterministic FSM,
(States; Inputs; Outputs; possible Updates; initial States)
- The first three elements are the same as for a deterministic FSM, but the last two are different.
- The form of the function possible Updates indicates there can be more than one next state and/or output valuation given a current state and input valuation.

Example: Nondeterministic model of pedestrians

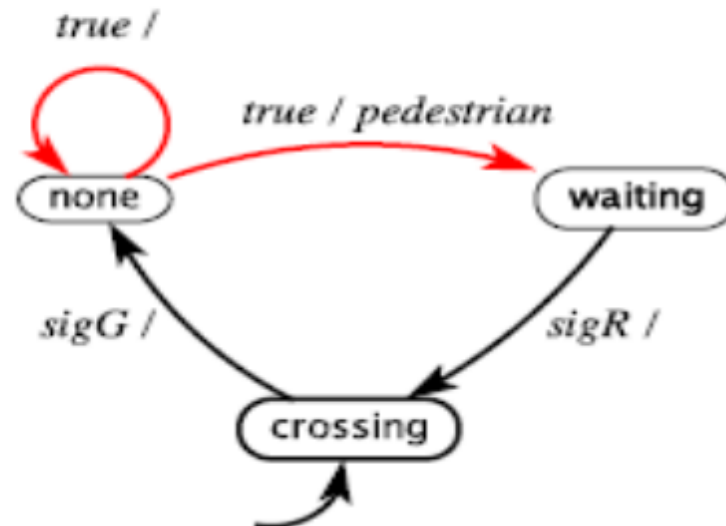
- The traffic light controller has one pure input signal, **pedestrian**.
- This input is present when a pedestrian arrives at the crosswalk.
- The traffic light will remain green unless a pedestrian arrives.
- Some other subsystem is responsible for generating the pedestrian event.
- That other subsystem is part of the environment of the FSM.
- A question becomes how to model the environment. In the traffic light example, we could construct a model of pedestrian flow

Cont.,

- The FSM in the following Figure models arrivals of pedestrians at a crosswalk with a traffic light controller

inputs: *sigR*, *sigG*, *sigY* : pure

outputs: *pedestrian* : pure



Cont.,

- The initial state is crossing.
- When sigG is received, the FSM transitions to none.
- Both transitions from this state have guard true, indicating that they are always enabled.
- Since both are enabled, this machine is nondeterministic.
- The FSM may stay in the same state and produce no output, or it may transition to waiting and produce pure output pedestrian.

$$\begin{aligned}
States &= \{\text{none}, \text{waiting}, \text{crossing}\} \\
Inputs &= (\{\text{sigG}, \text{sigY}, \text{sigR}\} \rightarrow \{\text{present}, \text{absent}\}) \\
Outputs &= (\{\text{pedestrian}\} \rightarrow \{\text{present}, \text{absent}\}) \\
initialStates &= \{\text{crossing}\}
\end{aligned}$$

The update relation is given below:

$$possibleUpdates(s, i) = \begin{cases} \{(\text{none}, \text{absent})\} & \text{if } s = \text{crossing} \\ & \wedge i(\text{sigG}) = \text{present} \\ \{(\text{none}, \text{absent}), (\text{waiting}, \text{present})\} & \text{if } s = \text{none} \\ \{(\text{crossing}, \text{absent})\} & \text{if } s = \text{waiting} \\ & \wedge i(\text{sigR}) = \text{present} \\ \{(s, \text{absent})\} & \text{otherwise} \end{cases}$$

Sheet

- Textbook Ch., 3